

Embedded Systems and Internet-of-Things - First Assignment

Kimi Osti

October 22, 2024

Abstract

The project is a game set to run on an Arduino UNO board, to which are attached five LEDs, four buttons, one potentiometer and a LCD screen. Before the game, when the board is booted, the player will see a welcome screen in which he will be able to set the difficulty (via the potentiometer). The game is set in rounds: every round, the LCD screen will show the player a number between 0 and 15, and the user has to convert it to binary using the buttons to light the LEDs corresponding to the bits that have value 1 in the binary representation. When the player is not able to correctly convert the number on time (the available time is reduced every round, according to the difficulty) a game over screen is shown, with the points achieved, and the game goes back to the welcome screen.

Contents

1	Problem Description	2
2	Domain Analysis	3
2.1	Device Analysis	3
2.2	Model Analysis	4
3	Project Architecture	6
3.1	Hardware Architecture	6
3.2	Software Architecture	7
3.2.1	Premises	7
3.2.2	Hardware Solutions	8
3.2.3	Code Solutions	8
3.2.4	Code Modularisation	9
4	System Demonstration	12

Chapter 1

Problem Description

The software must realise a simple input-output game based on the binary conversion of decimal numbers. When booted, the system must show a welcome message on the screen that reads "Welcome to GMB! Press B1 to start" while the red led is pulsing. If button B1 is not pressed within ten seconds, the system must go in a deep sleeping state (if any button is pressed it then wakes up), otherwise the actual game starts. The game is developed in rounds: each round a number in decimal base is shown on the screen, and the user has a given time to translate it to binary using the four green LEDs as bits (they are switched on and off by pressing the buttons). For a round to be successfully completed, the number has to be converted correctly by the end of the available time. After every successful round, the partial score is shown and time is reduced by some factor, determined according to the difficulty level, decided turning the potentiometer before starting the game. After an unsuccessful round, the red LED is turned on for 1 second, and then a game over message that says "Game Over! Final Score XXX" is shown for ten seconds, after which the system goes back to the initial state.

Chapter 2

Domain Analysis

The domain analysis can be naturally split in two parts considering the two main aspects of working with this kind of boards: the device analysis - focusing on the hardware parts and how to connect them - and the model analysis - focusing on the aspects that are implemented by the software.

2.1 Device Analysis

First off, we can schematise the devices that will be attached to our board, understanding their main functions:

- Output Devices

LCD Screen: used to show some messages to the player. It also gives him small instructions to continue in the game.

LEDs: are used with different meanings depending on the colour:

Red LED: used as a mere aesthetic device when fading, it also shows to the player that he has lost before his game over screen.

Four Green LEDs: used only during the rounds, they have the very important function of signalling to the player the number he composed in binary. In our analysis, L1 will represent the most significant bit and L4 will represent the least significant.

- Input Devices

Potentiometer: used only in the beginning of the game, it is used as an analogical device to input the level of difficulty.

Buttons: they are the main input device for the player, who will use them during the rounds to light the green LEDs and compose the number in binary base.

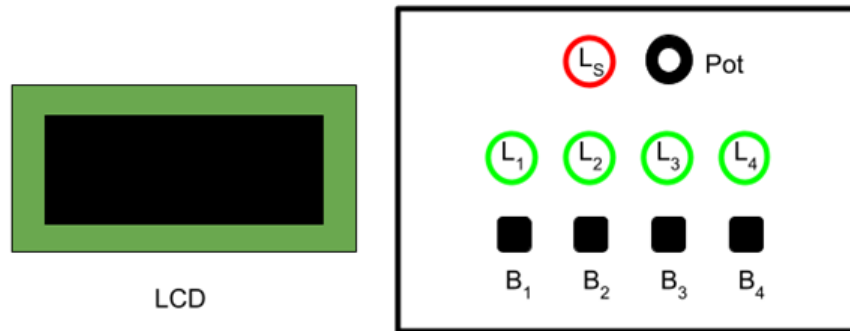


Figure 2.1: Approximate disposition of devices on the breadboard (taken from Professor's assignment)

2.2 Model Analysis

From a programming standpoint, the domain can intuitively be divided into the various states of the game:

- *New Game*: Before starting the game, the welcome screen has to be shown, the red LED has to pulse and just the button B1 has to be listened. After 10 seconds of inactivity, the system must go into the *Deep Sleeping* state, otherwise into the *Game Start* state.
- *Deep Sleeping*: the system is in low power consumption state, waiting for any button to be pressed. When this happens, it goes back to the *New Game* state.
- *Game Start*: round time is initialised to the default, score is set to zero, the LEDs go off, the LCD shows a "Go!" message and then the game goes to *Round* state. It's a transition state in which input is ignored.
- *Round*: it's the central state of the game, in which the player sees the number to convert on the LCD Screen and has to give an input thanks to all the four buttons. When the round time runs out, the game goes to *Round Over* state.
- *Round Over*: in this state, the game controls the state of the green LEDs. The green LEDs' state determines the binary value inserted by the player, and it is compared to the value shown on the LCD screen. If it's correct, the game goes to the *Show Score* state, otherwise it goes to the *Game Over* state.

- *Show Score*: The LCD Screen shows a message that shows the partial score the player has achieved, and the time for the next round is decreased according to the difficulty level. Then, after some time, the new round starts.
- *Game Over*: the red LED is turned on for one second, and then a game over message is shown on the LCD screen, before going back to the *New Game* state after 10 seconds.

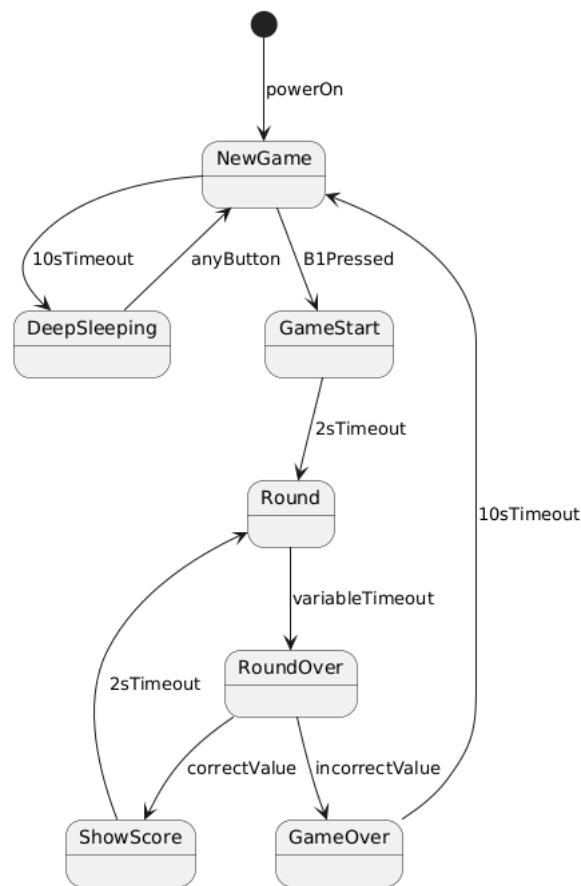


Figure 2.2: State diagram of the various stages of the game

Chapter 3

Project Architecture

The project will be run on an Arduino UNO board, and it will be programmed in C++.

3.1 Hardware Architecture

From a hardware standpoint, given the needed devices, the first step would be to create the circuit that will host them. This can be done with the UNO board, a breadboard and some jumpers.

LEDs A LED is a diode, that needs a tension to emit light. Simple as it is, its anode can be directly connected to one of the digital pins of the board, and the cathode will be connected to the ground. In our case, given that Arduino UNO has a VCC of 5V and the LEDs only need 2V, we can connect a 220Ohm resistor between LED and ground in order to lower the tension.

Buttons A button is a digital input device. So, it needs to be powered by the board's VCC and, on the other pole, to be connected to one of the board's digital pin. In order to give zero as a default value, a pull-down resistor of 10KOhm is added connecting the button to the ground, on the pole where the input pin is connected.

Potentiometer The potentiometer is an analogical input device. It has three pins: two lateral ones for VCC and ground, and a central one to which a jumper going to an analogical input pin will be connected.

LCD Screen The LCD screen communicates with the board thanks to the I^2C protocol. This means that it will have four pins: two for the serial

Accurately reading the diagram, it can also be noticed how the game flow is strongly time-oriented: many state transactions are dictated by timeouts of some kind.

Arduino only supports interrupts on two digital pins but - thanks to the `EnableInterrupt` library - button interrupts can be sent through any digital pin. This means that all four buttons can signal input to the MCU via interrupts.

To achieve the pulsing LED effect, we need to simulate analogical output. This can be done through some specific digital pins thanks to `PulseWidth Modulation`, that exploit timers to emulate an analogical output value. It can become problematic if timers are also used for timeout interrupts.

3.2.2 Hardware Solutions

The main issue is handling the digital pins.

The four buttons are attached to pins 2, 3, 4 and 5. This because 2 and 3 already support interrupts, so they are the most natural choice. On the other hand, pins 0 and 1 are preferably avoided because they serve asynchronous serial communication purposes.

The LCD Screen has the SCL line connected to pin A5, and the SDA line connected to pin A4.

The green LEDs are connected to pins 6, 7, 8 and 9 since they only use digital output functions.

The red LED, that pulses emulating analogical output, is connected to pin 10. This because it supports `PulseWidth Modulation`. It is based on timer 1, which is not problematic in this case because interrupt-driven timeouts aren't used.

3.2.3 Code Solutions

Event queue The game is strongly event-based. In this domain, button input and timeouts are considered events. When an event occurs, it is stored in a bounded queue. Considering that button input is important, but can affordably be lost and is very rare since it depends on a human operator, the boundary policy is that button input is ignored after reaching max occupation (which is very rare). On the other hand, timeouts can't be lost for any reason. Because of this, they are handled regardless of occupation.

Button input These events are handled through interrupts. This comes handy when the system is sleeping, and only interrupts can wake it up. It was previously disclosed how to attach interrupts to four digital pins, exploiting

the EnableInterrupt library. Also, bouncing has to be accounted for. To avoid it, interrupt timestamps are stored and, if two interrupts happen in a given amount of time, the second one is ignored. Said amount of time has to be measured to avoid both bouncing and event loss. Event loss, though, is not really an issue in a human-operated system since bouncing can be avoided waiting just a few milliseconds.

Timeouts These are the most important events in the game, and they can't be lost, because that would mean possibly waiting for another timeout event to switch state. This is the main reason because of which queue limitation can be exceeded with timeouts. Since Arduino UNO does not support interrupt nesting, handling timeouts with interrupts can become problematic: if a timeout happens while a button pressure is being handled it is lost. Because of this, and because of other problems (mainly related to the TimerOne library) I chose to use the Timer library exploiting a utility class, handling time solely in the main loop. Furthermore, in this system it would not be so problematic if a timeout is recorded some fraction of a second later, if compared to possible timeout loss.

Main Loop Given that the program relies on an event queue to handle behaviour, the main loop function will - each cycle - fetch an event from the queue, interpret it according to the state in which the game is, and act accordingly. When the queue is empty, a fictitious event - which is always ignored - is returned from the queue. Since the main loop is active in every state (except from the deep sleeping state, in which though timeouts can't happen) it is not really problematic to poll the timer each iteration. Since timeouts are directly detected by the main loop, it's not explicitly handled by the queue, that is here restricted to the sole handling of button interrupts.

3.2.4 Code Modularisation

In order to keep the main file as simple as possible, and to keep the main loop and the setup functions understandable, code is divided in modules. This can be done creating a few utility libraries.

state_utils.h

This library will have all the functions needed to handle the states. So, it will declare an enum with all the possible states, that is reused also by the main loop function. It will also implement two functions:

- *switch_state(state)* changes the actual state to the next, indicated as a parameter, and returns the timeout for that state.
- *check_round()* checks whether the number inserted by the user is correct.

events_utils.h

This library will define an enum type to classify interrupts, a struct type for queue nodes, and the functions to handle it.

- *queue_init()* initialises the event queue.
- *add_event(event)* adds an event to the queue.
- *get_event()* returns an event from the queue.
- *clear_queue()* clears the event queue.

led_utils.h

This library will implement all the features related to LEDs:

- *led_init()* sets up all the LED output pins.
- *turn_led(int)* given a pin number, turns the LED attached to it.
- *sweep_leds()* turns off all the LEDs.
- *get_led_state(int)* returns the actual state of a LED.
- *fade_led()* fades the red LED.

but_utils.h

This library will implement all the features related to buttons:

- *but_init()* initialises the buttons, attaching interrupt handlers.

lcd_utils.h

This library will implement all the features related to the LCD screen:

- *lcd_init()* initialises the LCD Screen.
- *welcome_screen()* shows the welcome message on the LCD Screen.
- *go_screen()* shows the pre-game message on the LCD Screen.

- *show_number(int)* shows the number that the player has to convert on the LCD Screen.
- *score_screen(int)* shows the score between rounds on the LCD Screen.
- *game_over_screen(int)* shows the final message on the LCD Screen.
- *lcd_turn_off()* clears the screen and turns off the backlight. It's called when the system is sleeping.

Note on modularisation The ones listed above are libraries exposing public functions: each module, in its own implementation, will define and use some other private functions.

loop() Given an event from the queue, according to its type and the state in which the system is, the main loop will forward requests of behaviour to libraries, asking to control output devices or to switch states accordingly. Some states, though, require some action even when no events are happening (such as the pulsing LED of the initial state). This has to be implemented in the main loop, which won't be purely event-oriented but will always run. Also, some events not purely linked to timeouts (such as the pulsing of the LED) have to be handled with time measuring instruments. This is also one of the reasons why I decided to rely on the Timer library rather than on interrupt-driven timers.

Chapter 4

System Demonstration

This is a brief demonstration of the system functioning, walking through all the possible game states.